argument passing. This works on one level, but it isn't really the whole story. The block is effectively an entire other method that can be invoked or passed forward as an argument to another method. For example:

intro/block_example.rb

```
def call_block
  puts "Start of method"
  yield
  yield
  puts "End of method"
end

call_block { puts "In the block" }
```

produces:

```
Start of method
In the block
In the block
End of method
```

In this example, code in the block (puts "In the block") is executed twice, once for each call to yield passing control to the block.

You can provide arguments to yield, and they will be passed to the block. Within the block, you list the names of the parameters to receive these arguments between vertical bars (|params...|). The following example shows a method calling its associated block twice, passing the block two arguments each time:

intro/block_example2.rb

```
def who_says_what
  yield("Dave", "hello")
  yield("Andy", "goodbye")
end

who_says_what { |person, phrase| puts "#{person} says #{phrase}" }
```

produces:

```
Dave says hello
Andy says goodbye
```

You can use code blocks to package code to implement a later callback. Code blocks can be used to pass around chunks of code, and are used throughout the Ruby standard library to allow methods to perform an action on successive elements from a collection such as an array. The act of doing something similar to all objects in a collection is called *enumeration* in Ruby; other languages call this *iteration*.

```
animals = ["ant", "bee", "cat", "dog"]    # create an array
animals.each { |animal| puts animal }   # iterate over the contents
```

produces:

```
ant
bee
cat
dog
```